Policy Gradient: Reinforce

Kianté Brantley
CS 1840 Fall 2026

Announcements

HW4 is out and due 11/12 (this Thursday)!

Midterm resubmissions to improve your grade!

Check the course website for Office Hours:
https://harvard-cs-1840.github.io/2026/

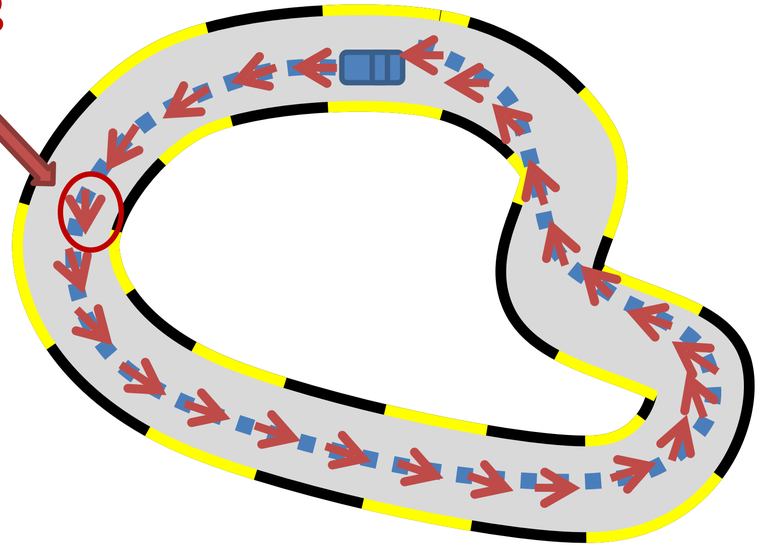
[Ross11a]

DAgger: Dataset Aggregation

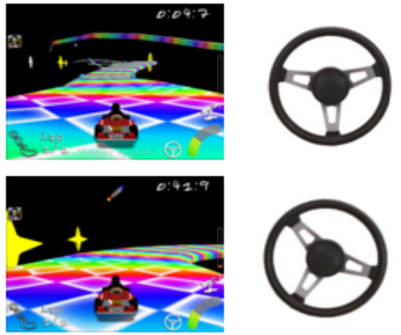
nth iteration

Execute π_{n-1} and Query Expert

Steering from expert

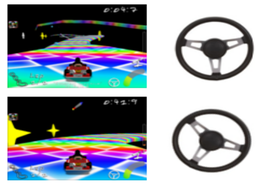


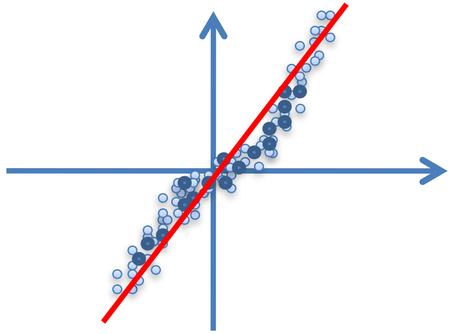
New Data



Aggregate Dataset

All previous data





Supervised Learning

New policy π_n

Finishing Up From Last Lecture

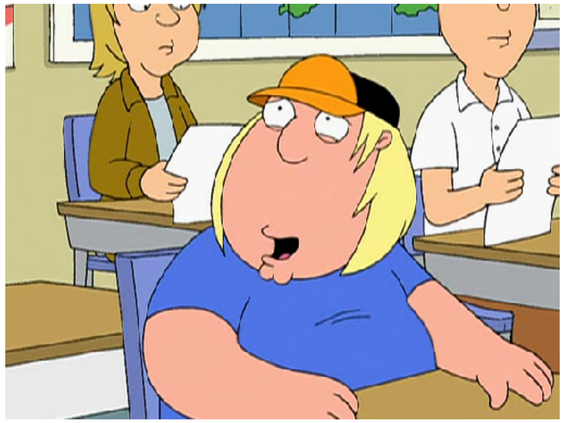


1. The DAgger (Data Aggregation) Algorithm

2. Analysis of DAgger: DAgger as online learning

Online Learning

Learner



convex Decision set Θ

Adversary

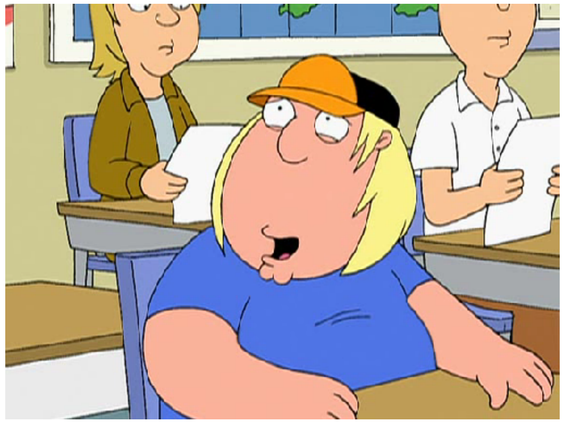


...

[Vovk92,Warmuth94,Freund97,Zinkevich03,Kalai05,Hazan06,Kakade08]

Online Learning

Learner



convex Decision set Θ

Learner picks a decision θ_0



Adversary

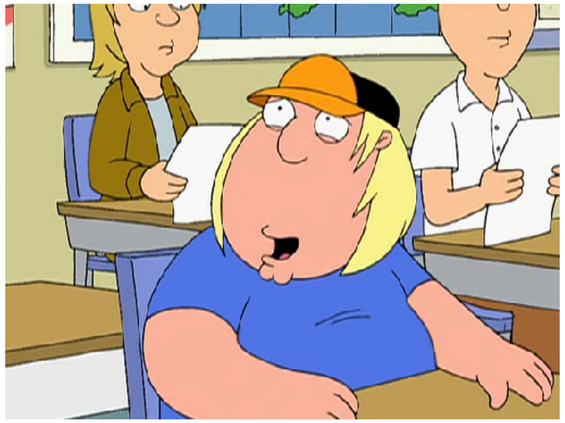


…

[Vovk92,Warmuth94,Freund97,Zinkevich03,Kalai05,Hazan06,Kakade08]

Online Learning

Learner



convex Decision set Θ

Learner picks a decision θ_0 →

← Adversary picks a loss $\ell_0 : \Theta \to \mathbb{R}$

Adversary

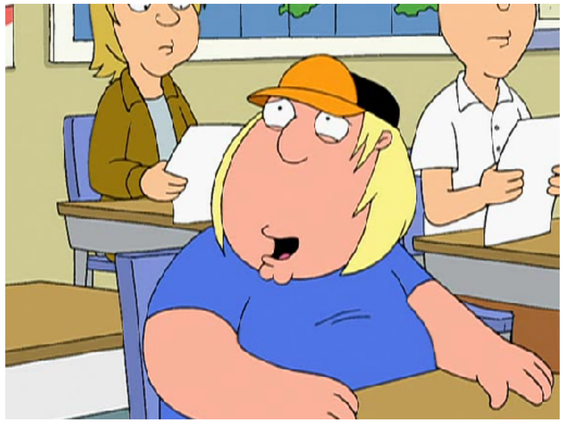


…

[Vovk92,Warmuth94,Freund97,Zinkevich03,Kalai05,Hazan06,Kakade08]

Online Learning

Learner



convex Decision set Θ

Learner picks a decision θ_0 →

Adversary picks a loss $\ell_0 : \Theta \to \mathbb{R}$ ←

Learner picks a new decision θ_1 →

…

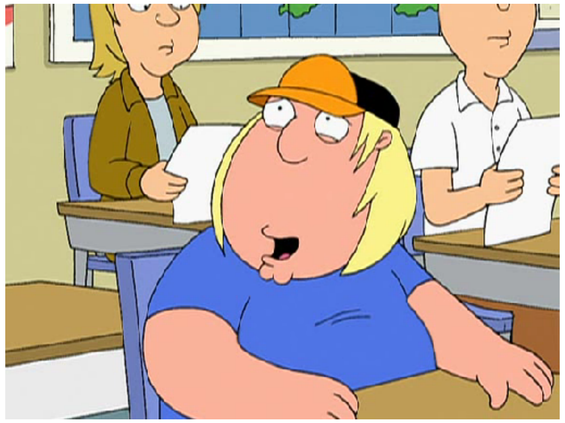
Adversary



[Vovk92,Warmuth94,Freund97,Zinkevich03,Kalai05,Hazan06,Kakade08]

Online Learning

Learner



convex Decision set Θ

Learner picks a decision θ_0 →

← Adversary picks a loss $\ell_0 : \Theta \to \mathbb{R}$

Learner picks a new decision θ_1 →

← Adversary picks a loss $\ell_1 : \Theta \to \mathbb{R}$

…

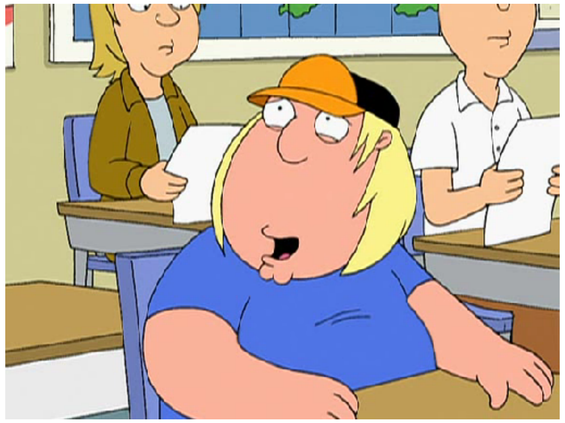
Adversary



[Vovk92,Warmuth94,Freund97,Zinkevich03,Kalai05,Hazan06,Kakade08]

Online Learning

Learner



convex Decision set Θ

Learner picks a decision θ_0

Adversary picks a loss $\ell_0 : \Theta \to \mathbb{R}$

Learner picks a new decision θ_1

Adversary picks a loss $\ell_1 : \Theta \to \mathbb{R}$

...

Adversary



$$\text{Regret} = \sum_{t=0}^{T-1} \ell_t(\theta_t) - \min_{\theta \in \Theta} \sum_{t=0}^{T-1} \ell_t(\theta)$$

[Vovk92,Warmuth94,Freund97,Zinkevich03,Kalai05,Hazan06,Kakade08]

Example: online linear regression

Can we perform linear regression online with non i.i.d (or even adversary) data?

Example: online linear regression

Can we perform linear regression online with non i.i.d (or even adversary) data?

Every iteration t :

Example: online linear regression

Can we perform linear regression online with non i.i.d (or even adversary) data?

Every iteration t :

1. Learner first picks $\theta_t \in$ Ball $\subset \mathbb{R}^d$

Example: online linear regression

Can we perform linear regression online with non i.i.d (or even adversary) data?

Every iteration t :

1. Learner first picks $\theta_t \in$ Ball $\subset \mathbb{R}^d$

2. Adversary **then** picks $x_t \in \mathcal{X} \subset \mathbb{R}^d, y_t \in [a, b]$

Example: online linear regression

Can we perform linear regression online with non i.i.d (or even adversary) data?

Every iteration t :

1. Learner first picks $\theta_t \in$ Ball $\subset \mathbb{R}^d$

2. Adversary **then** picks $x_t \in \mathcal{X} \subset \mathbb{R}^d, y_t \in [a, b]$

3. Learner suffers loss $\ell_t(\theta_t) = (\theta_t^\top x_t - y_t)^2$

Example: online linear regression

Can we perform linear regression online with non i.i.d (or even adversary) data?

Every iteration t :

1. Learner first picks $\theta_t \in \text{Ball} \subset \mathbb{R}^d$

2. Adversary **then** picks $x_t \in \mathcal{X} \subset \mathbb{R}^d, y_t \in [a, b]$

3. Learner suffers loss $\ell_t(\theta_t) = (\theta_t^\top x_t - y_t)^2$

Learner has to make decision θ_t based on history up to $t-1$,
while adversary could pick (x_t, y_t) even after seeing θ_t

Example: online linear regression

Can we perform linear regression online with non i.i.d (or even adversary) data?

Every iteration t :

1. Learner first picks $\theta_t \in \text{Ball} \subset \mathbb{R}^d$

2. Adversary **then** picks $x_t \in \mathcal{X} \subset \mathbb{R}^d, y_t \in [a, b]$

3. Learner suffers loss $\ell_t(\theta_t) = (\theta_t^\top x_t - y_t)^2$

Learner has to make decision θ_t based on history up to $t-1$,
while adversary could pick (x_t, y_t) even after seeing θ_t

Adversary seems too powerful…

Example: online linear regression

BUT, a very intuitive algorithm actually achieves no-regret property:

Example: online linear regression

BUT, a very intuitive algorithm actually achieves no-regret property:

Every iteration t :

1. Learner first picks θ_t that minimizes the aggregated loss

$$\theta_t = \arg \min_{\theta \in \text{Ball}} \sum_{i=0}^{t-1} \left(\theta^\top x_i - y_i\right)^2 + \lambda \|\theta\|_2^2$$

Example: online linear regression

BUT, a very intuitive algorithm actually achieves no-regret property:

Every iteration t :

1. Learner first picks θ_t that minimizes the aggregated loss

$$\theta_t = \arg \min_{\theta \in \text{Ball}} \sum_{i=0}^{t-1} \left(\theta^\top x_i - y_i\right)^2 + \lambda \|\theta\|_2^2$$

This is called Follow-the-Regularized-Leader (FTRL), and it achieves no-regret property:

Example: online linear regression

BUT, a very intuitive algorithm actually achieves no-regret property:

Every iteration t :

1. Learner first picks θ_t that minimizes the aggregated loss

$$\theta_t = \arg \min_{\theta \in \text{Ball}} \sum_{i=0}^{t-1} \left(\theta^\top x_i - y_i\right)^2 + \lambda \|\theta\|_2^2$$

This is called Follow-the-Regularized-Leader (FTRL), and it achieves no-regret property:

$$\sum_{i=0}^{T-1} \ell_i(\theta_i) - \min_{\theta \in \text{Ball}} \sum_{i=0}^{T-1} \ell_i(\theta) = O\left(\sqrt{T}\right)$$

Generally, Follow-the-Regularized-Leader is no-regret

At time step t, learner has seen $\ell_0, \ldots \ell_{t-1}$, which new decision she could pick?

$$\textbf{FTRL: } \theta_t = \min_{\theta \in \Theta} \sum_{i=0}^{t-1} \ell_i(\theta) + \lambda R(\theta)$$

Generally, Follow-the-Regularized-Leader is no-regret

At time step t, learner has seen $\ell_0, \ldots \ell_{t-1}$, which new decision she could pick?

$$\textbf{FTRL: } \theta_t = \min_{\theta \in \Theta} \sum_{i=0}^{t-1} \ell_i(\theta) + \lambda R(\theta)$$

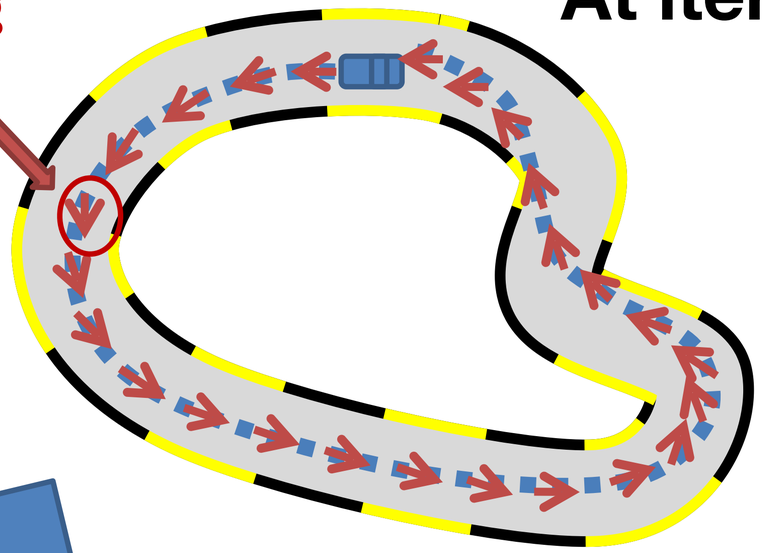
Informal Theorem (FTRL): when things are convex, FTRL is no-regret, i.e.,

$$\frac{1}{T}\left[\sum_{t=0}^{T-1} \ell_t(\theta_t) - \min_{\theta \in \Theta} \sum_{t=0}^{T-1} \ell_t(\theta)\right] = O\left(1/\sqrt{T}\right)$$

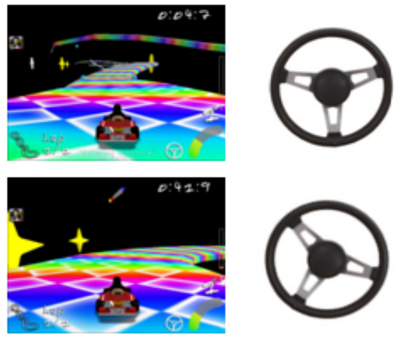
DAgger Revisit

At iteration t:

Steering from expert

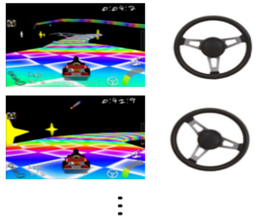


New Data

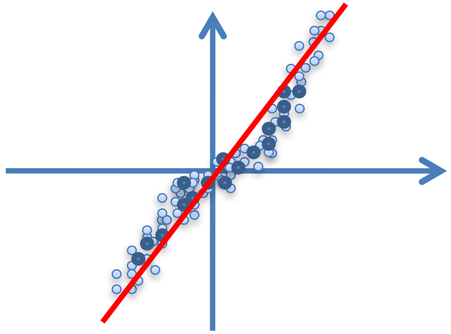


Aggregate Dataset

All previous data



New policy π_n

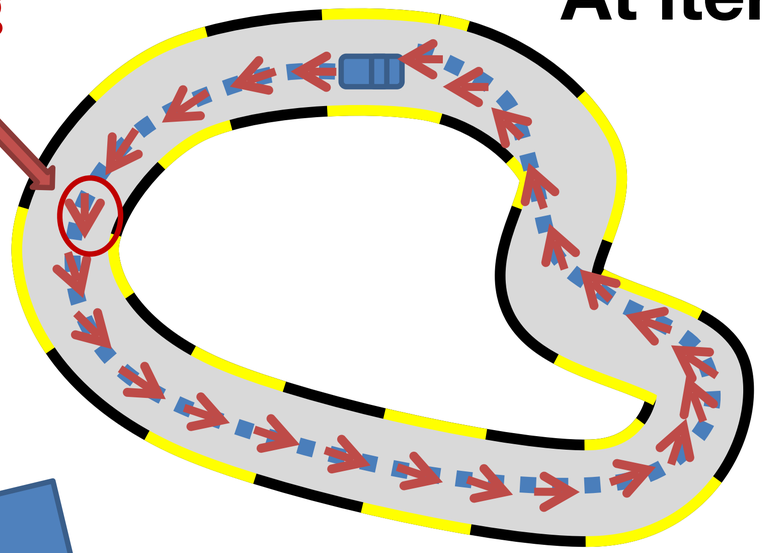


Supervised Learning

DAgger Revisit

At iteration t:

Steering from expert

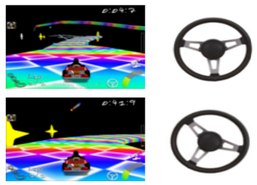


New Data

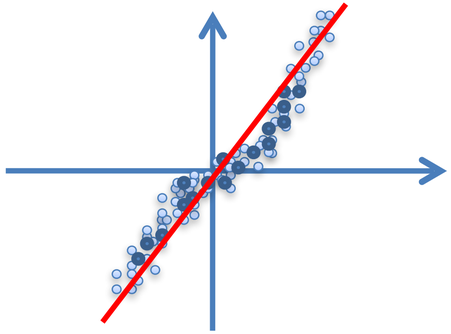
$$\ell_t(\pi) = \sum_{i=1}^{m} \|\pi(s^i) - \pi^\star(s^i)\|_2^2$$

Aggregate Dataset

All previous data



New policy π_n

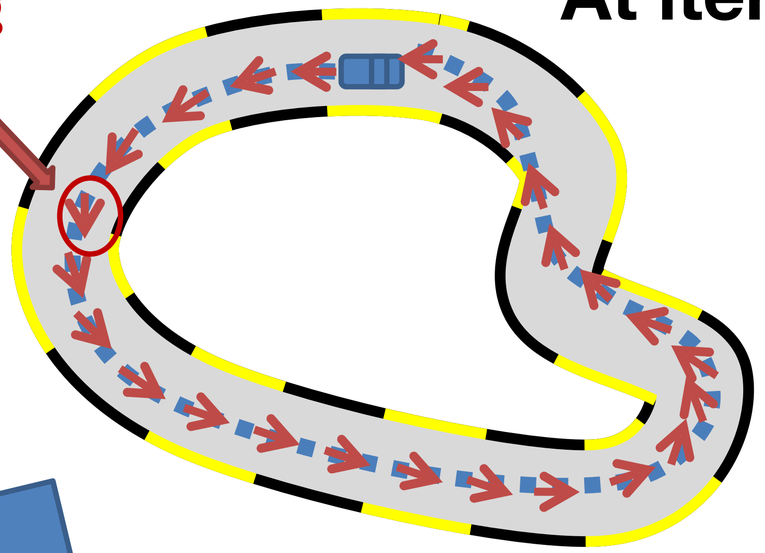


Supervised Learning

DAgger Revisit

At iteration t:

Steering from expert



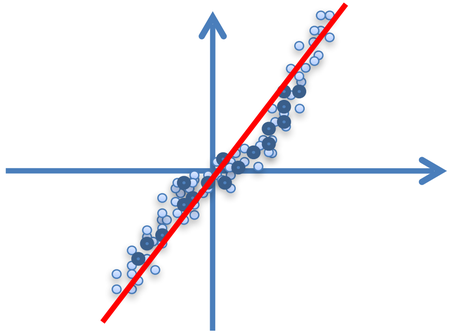
New Data

$$\ell_t(\pi) = \sum_{i=1}^{m} \|\pi(s^i) - \pi^\star(s^i)\|_2^2$$

Aggregate Dataset

All previous data

$$\sum_{i=0}^{t} \ell_i(\pi)$$



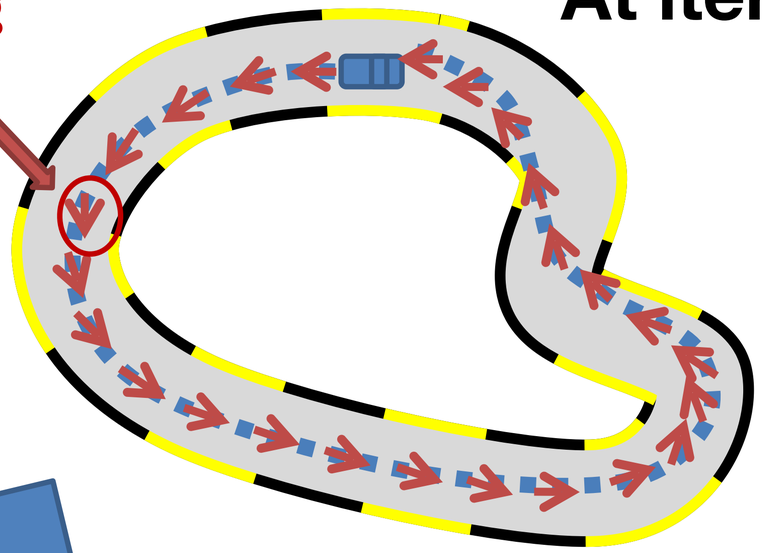
New policy π_n

Supervised Learning

DAgger Revisit

At iteration t:

Steering from expert



New Data

$$\ell_t(\pi) = \sum_{i=1}^{m} \|\pi(s^i) - \pi^\star(s^i)\|_2^2$$

Aggregate Dataset

All previous data

$$\sum_{i=0}^{t} \ell_i(\pi)$$

New policy π_n

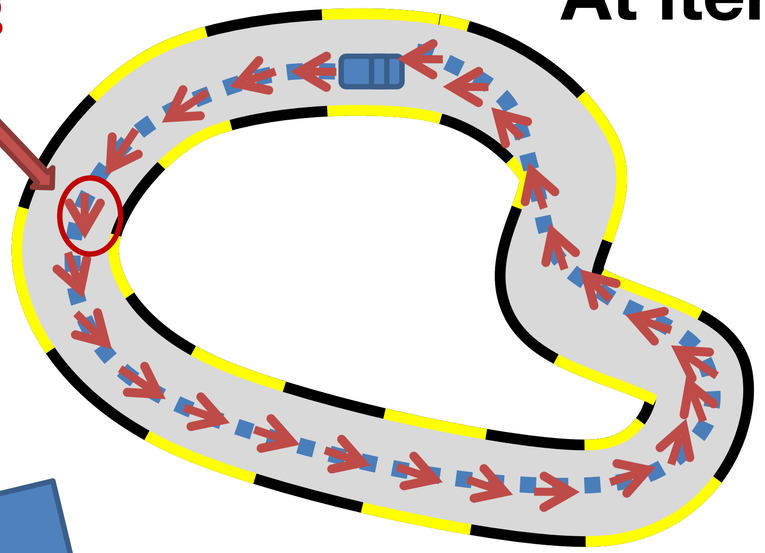
$$\pi_t = \arg\min_{\pi} \sum_{i=0}^{t} \ell_i(\pi) + \lambda R(\pi)$$

Supervised Learning

DAgger Revisit

At iteration t:

Steering from expert



New Data

$$\ell_t(\pi) = \sum_{i=1}^{m} \|\pi(s^i) - \pi^\star(s^i)\|_2^2$$

Aggregate Dataset

All previous data

$$\sum_{i=0}^{t} \ell_i(\pi)$$

New policy π_n

$$\pi_t = \arg\min_{\pi} \sum_{i=0}^{t} \ell_i(\pi) + \lambda R(\pi)$$

Supervised Learning

Data Aggregation = Follow-the-Regularized-Leader Online Learner

Dagger Algorithm:

1. The DAgger algorithm

Initialize π^0, and dataset $\mathscr{D} = \varnothing$

For $t = 0 \rightarrow T - 1$:

1. W/ π^t, generate dataset $\mathscr{D}^t = \{s_i, a_i^\star\}, s_i \sim d_\mu^{\pi^t}, a_i^\star = \pi^\star(s_i)$
2. **Data aggregation**: $\mathscr{D} = \mathscr{D} + \mathscr{D}^t$
3. **Update policy via Supervised-Learning**: $\pi^{t+1} = \mathrm{SL}\left(\mathscr{D}\right)$

Dagger Algorithm:

1. The DAgger algorithm

Initialize π^0, and dataset $\mathscr{D} = \varnothing$

For $t = 0 \rightarrow T - 1$:

1. W/ π^t, generate dataset $\mathscr{D}^t = \{s_i, a_i^\star\}, s_i \sim d_\mu^{\pi^t}, a_i^\star = \pi^\star(s_i)$
2. **Data aggregation**: $\mathscr{D} = \mathscr{D} + \mathscr{D}^t$
3. **Update policy via Supervised-Learning**: $\pi^{t+1} = \mathrm{SL}\left(\mathscr{D}\right)$

2. We can see that DAgger is essentially an online-learning algorithm (FTRL)

Recap: two definitions of MDPs

$$\mathscr{M} = \{P, r, \gamma, \mu, S, A\}$$

where $s_0 \sim \mu$

$$\text{Objective: } J(\pi) := \mathbb{E}\left[\sum_{h=0}^{\infty} \gamma^h r(s_h, a_h)\right]$$

Recap: two definitions of MDPs

$$\mathscr{M} = \{P, r, \gamma, \mu, S, A\}$$

where $s_0 \sim \mu$

$$\text{Objective: } J(\pi) := \mathbb{E}\left[\sum_{h=0}^{\infty} \gamma^h r(s_h, a_h)\right]$$

$$\mathscr{M} = \{P, r, H, \mu, S, A\}$$

where $s_0 \sim \mu$

$$\text{Objective: } J(\pi) := \mathbb{E}\left[\sum_{h=0}^{H-1} r(s_h, a_h)\right]$$

Recap: value-based learning

Deep Q network (DQN)

$$\theta \Leftarrow \theta - \eta \frac{1}{|\mathcal{B}|} \sum_{s,a,r,s' \in \mathcal{B}} \left(Q_\theta(s,a) - r - \gamma \max_{a'} Q_{\tilde{\theta}}(s',a') \right) \nabla_\theta Q_\theta(s,a)$$

Recap: value-based learning

Deep Q network (DQN)

$$\theta \Leftarrow \theta - \eta \frac{1}{|\mathcal{B}|} \sum_{s,a,r,s' \in \mathcal{B}} \left(Q_\theta(s,a) - r - \gamma \max_{a'} Q_{\tilde{\theta}}(s',a') \right) \nabla_\theta Q_\theta(s,a)$$

Avg over a mini-batch
sampled from a replay buffer

Recap: value-based learning

Deep Q network (DQN)

$$\theta \Leftarrow \theta - \eta \frac{1}{|\mathcal{B}|} \sum_{s,a,r,s' \in \mathcal{B}} \left(Q_\theta(s,a) - r - \gamma \max_{a'} Q_{\tilde{\theta}}(s',a') \right) \nabla_\theta Q_\theta(s,a)$$

Avg over a mini-batch sampled from a replay buffer

Target network being updated once every C steps of gradient updates

Recap: value-based learning

Issues of Q-learning

1. In worst case, it **cannot** even guarantee convergence when function approximator is used

Recap: value-based learning

Issues of Q-learning

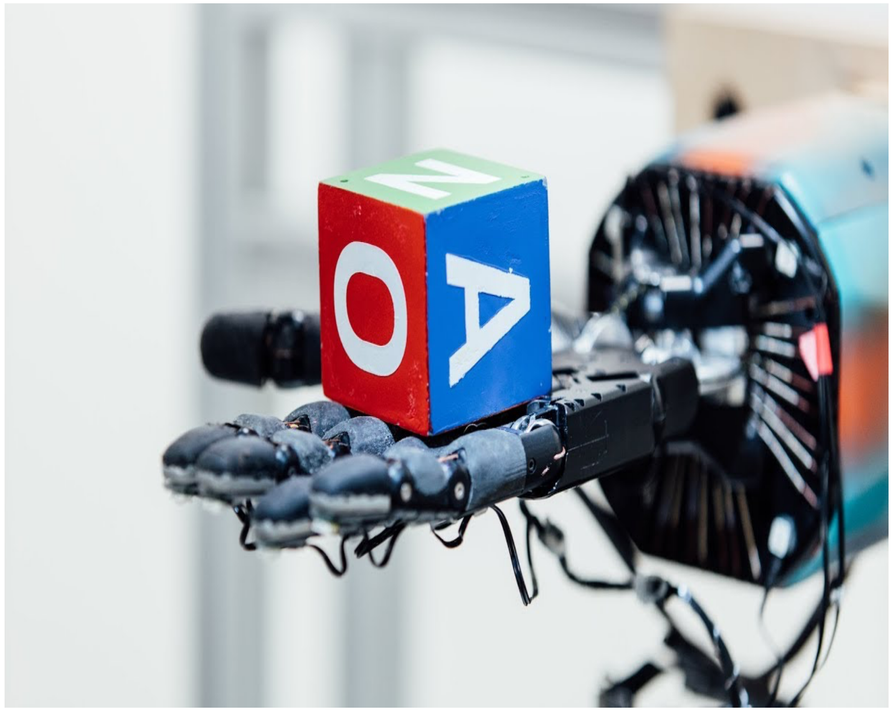
1. In worst case, it **cannot** even guarantee convergence when function approximator is used

2. Q-learning is not a direct approach — It does not **directly** optimize the ultimate objective: maximizing expected total reward

Today

Reinforce: policy gradient algorithm that can directly optimizes the ultimate objective

Policy gradient is very popular in practice



Robotics

ChatGPT

Generative AI

Outline:

1. Warm up: computing gradient using importance weighting

2. Policy Gradient formulation

Warm Up: Importance Weighting

$$J(\theta) = \mathbb{E}_{x \sim P_\theta} \left[f(x) \right]$$

Warm Up: Importance Weighting

$$J(\theta) = \mathbb{E}_{x \sim P_\theta}\left[f(x)\right] \qquad \nabla_\theta J(\theta) = \nabla_\theta \mathbb{E}_{x \sim P_\theta} f(x)$$

Warm Up: Importance Weighting

$$J(\theta) = \mathbb{E}_{x \sim P_\theta}\left[f(x)\right] \qquad \nabla_\theta J(\theta) = \nabla_\theta \mathbb{E}_{x \sim P_\theta} f(x)$$

Suppose that I have a sampling distribution ρ, s.t., $\max_x P_\theta(x)/\rho(x) < \infty$

Warm Up: Importance Weighting

$$J(\theta) = \mathbb{E}_{x \sim P_\theta}\left[f(x)\right] \qquad \nabla_\theta J(\theta) = \nabla_\theta \mathbb{E}_{x \sim P_\theta} f(x)$$

Suppose that I have a sampling distribution ρ, s.t., $\max_x P_\theta(x)/\rho(x) < \infty$

$$\nabla_\theta J(\theta) = \nabla_\theta \mathbb{E}_{x \sim P_\theta} f(x) = \nabla_\theta \mathbb{E}_{x \sim \rho} \frac{P_\theta(x)}{\rho(x)} f(x)$$

Warm Up: Importance Weighting

$$J(\theta) = \mathbb{E}_{x \sim P_\theta}\left[f(x)\right] \qquad \nabla_\theta J(\theta) = \nabla_\theta \mathbb{E}_{x \sim P_\theta} f(x)$$

Suppose that I have a sampling distribution ρ, s.t., $\max_x P_\theta(x)/\rho(x) < \infty$

$$\nabla_\theta J(\theta) = \nabla_\theta \mathbb{E}_{x \sim P_\theta} f(x) = \nabla_\theta \mathbb{E}_{x \sim \rho} \frac{P_\theta(x)}{\rho(x)} f(x) \quad = \mathbb{E}_{x \sim \rho} \frac{\nabla_\theta P_\theta(x)}{\rho(x)} f(x)$$

Warm Up: Importance Weighting

$$J(\theta) = \mathbb{E}_{x \sim P_\theta}\left[f(x)\right] \qquad \nabla_\theta J(\theta) = \nabla_\theta \mathbb{E}_{x \sim P_\theta} f(x)$$

Suppose that I have a sampling distribution ρ, s.t., $\max_x P_\theta(x)/\rho(x) < \infty$

$$\nabla_\theta J(\theta) = \nabla_\theta \mathbb{E}_{x \sim P_\theta} f(x) = \nabla_\theta \mathbb{E}_{x \sim \rho} \frac{P_\theta(x)}{\rho(x)} f(x) \quad = \mathbb{E}_{x \sim \rho} \frac{\nabla_\theta P_\theta(x)}{\rho(x)} f(x) \quad \approx \frac{1}{N} \sum_{i=1}^{N} \frac{\nabla_\theta P_\theta(x_i)}{\rho(x_i)} f(x_i)$$

Warm Up: Importance Weighting

$$J(\theta) = \mathbb{E}_{x \sim P_\theta}\left[f(x)\right] \qquad \nabla_\theta J(\theta) = \nabla_\theta \mathbb{E}_{x \sim P_\theta} f(x)$$

Suppose that I have a sampling distribution ρ, s.t., $\max_x P_\theta(x)/\rho(x) < \infty$

$$\nabla_\theta J(\theta) = \nabla_\theta \mathbb{E}_{x \sim P_\theta} f(x) = \nabla_\theta \mathbb{E}_{x \sim \rho} \frac{P_\theta(x)}{\rho(x)} f(x) \quad = \mathbb{E}_{x \sim \rho} \frac{\nabla_\theta P_\theta(x)}{\rho(x)} f(x) \quad \approx \frac{1}{N} \sum_{i=1}^{N} \frac{\nabla_\theta P_\theta(x_i)}{\rho(x_i)} f(x_i)$$

To compute gradient at θ_0: $\nabla_\theta J(\theta_0)$ (in short of $\nabla_\theta J(\theta)\,|_{\theta=\theta_0}$)

Warm Up: Importance Weighting

$$J(\theta) = \mathbb{E}_{x \sim P_\theta}\left[f(x)\right] \qquad \nabla_\theta J(\theta) = \nabla_\theta \mathbb{E}_{x \sim P_\theta} f(x)$$

Suppose that I have a sampling distribution ρ, s.t., $\max_x P_\theta(x)/\rho(x) < \infty$

$$\nabla_\theta J(\theta) = \nabla_\theta \mathbb{E}_{x \sim P_\theta} f(x) = \nabla_\theta \mathbb{E}_{x \sim \rho} \frac{P_\theta(x)}{\rho(x)} f(x) \quad = \mathbb{E}_{x \sim \rho} \frac{\nabla_\theta P_\theta(x)}{\rho(x)} f(x) \quad \approx \frac{1}{N} \sum_{i=1}^{N} \frac{\nabla_\theta P_\theta(x_i)}{\rho(x_i)} f(x_i)$$

To compute gradient at θ_0: $\nabla_\theta J(\theta_0)$ (in short of $\nabla_\theta J(\theta)\,|_{\theta=\theta_0}$)

We can set sampling distribution $\rho = P_{\theta_0}$

Warm Up: Importance Weighting

$$J(\theta) = \mathbb{E}_{x\sim P_\theta}\left[f(x)\right] \qquad \nabla_\theta J(\theta) = \nabla_\theta \mathbb{E}_{x\sim P_\theta} f(x)$$

Suppose that I have a sampling distribution ρ, s.t., $\max_x P_\theta(x)/\rho(x) < \infty$

$$\nabla_\theta J(\theta) = \nabla_\theta \mathbb{E}_{x\sim P_\theta} f(x) = \nabla_\theta \mathbb{E}_{x\sim\rho} \frac{P_\theta(x)}{\rho(x)} f(x) \quad = \mathbb{E}_{x\sim\rho} \frac{\nabla_\theta P_\theta(x)}{\rho(x)} f(x) \quad \approx \frac{1}{N}\sum_{i=1}^{N} \frac{\nabla_\theta P_\theta(x_i)}{\rho(x_i)} f(x_i)$$

To compute gradient at θ_0: $\nabla_\theta J(\theta_0)$ (in short of $\nabla_\theta J(\theta)\,|_{\theta=\theta_0}$)

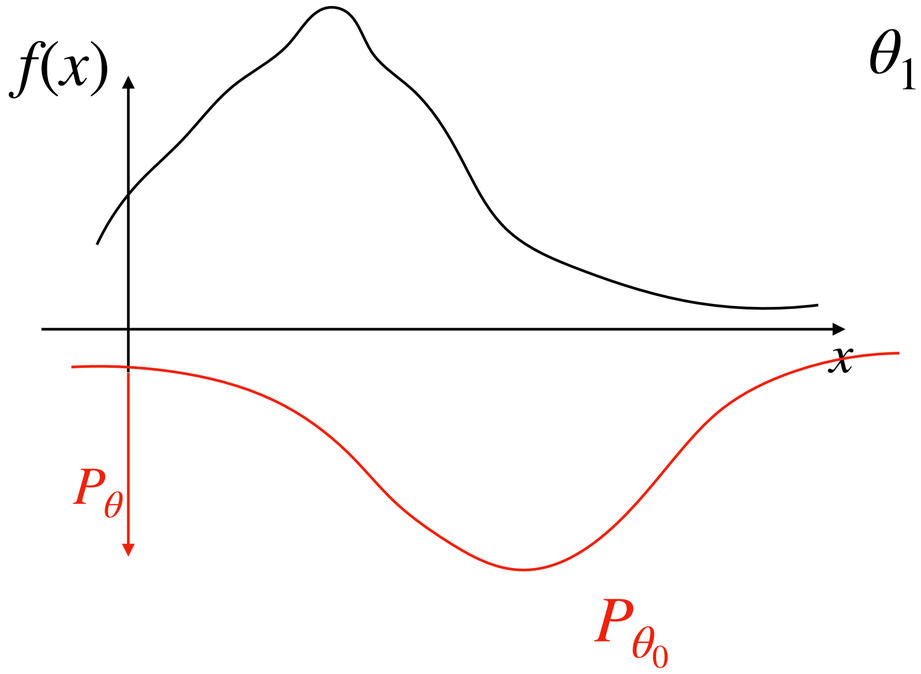
We can set sampling distribution $\rho = P_{\theta_0}$

$$\nabla_\theta J(\theta_0) = \mathbb{E}_{x\sim P_{\theta_0}}\left[\nabla_\theta \ln P_{\theta_0}(x) \cdot f(x)\right]$$

Warm Up:

$$\nabla_\theta J(\theta)\,|_{\theta=\theta_0} = \mathbb{E}_{x\sim P_{\theta_0}} \nabla_\theta \ln P_{\theta_0}(x) \cdot f(x)$$

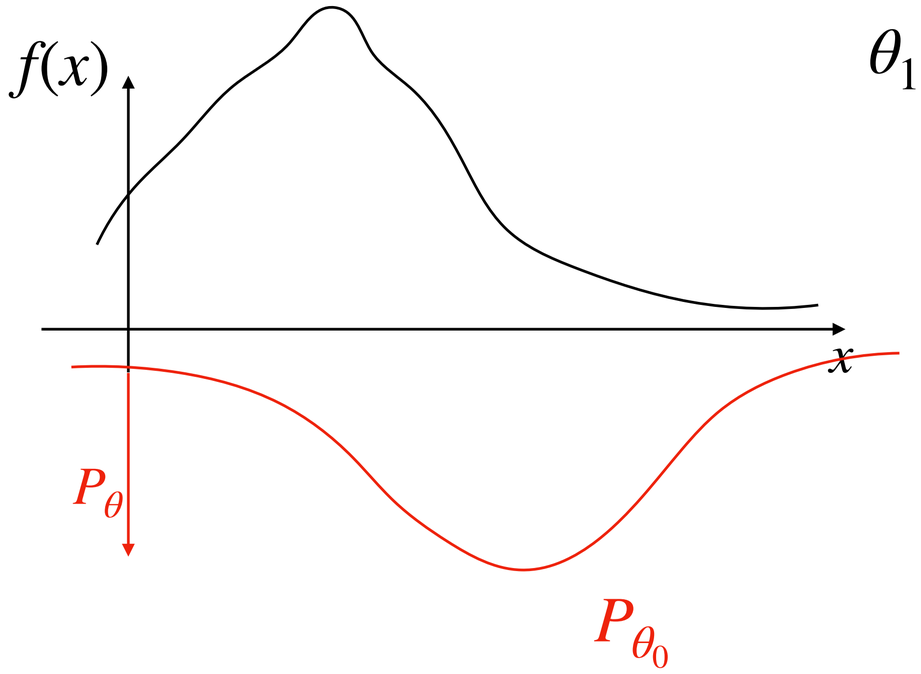
$$\theta_1 = \theta_0 + \eta \nabla_\theta J(\theta_0)$$



Warm Up:

$$\nabla_\theta J(\theta)\,|_{\theta=\theta_0} = \mathbb{E}_{x\sim P_{\theta_0}} \nabla_\theta \ln P_{\theta_0}(x) \cdot f(x)$$

$$\theta_1 = \theta_0 + \eta \nabla_\theta J(\theta_0)$$

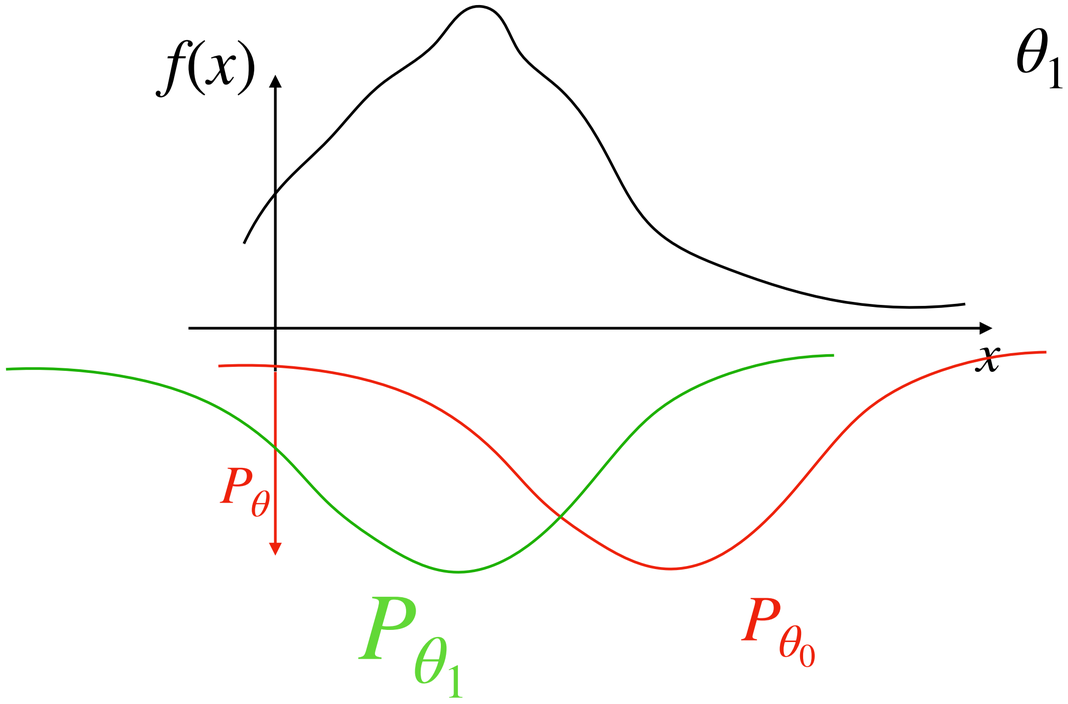


Q: how would this distribution move during the update?

Warm Up:

$$\nabla_\theta J(\theta)\,|_{\theta=\theta_0} = \mathbb{E}_{x\sim P_{\theta_0}} \nabla_\theta \ln P_{\theta_0}(x) \cdot f(x)$$

$$\theta_1 = \theta_0 + \eta \nabla_\theta J(\theta_0)$$

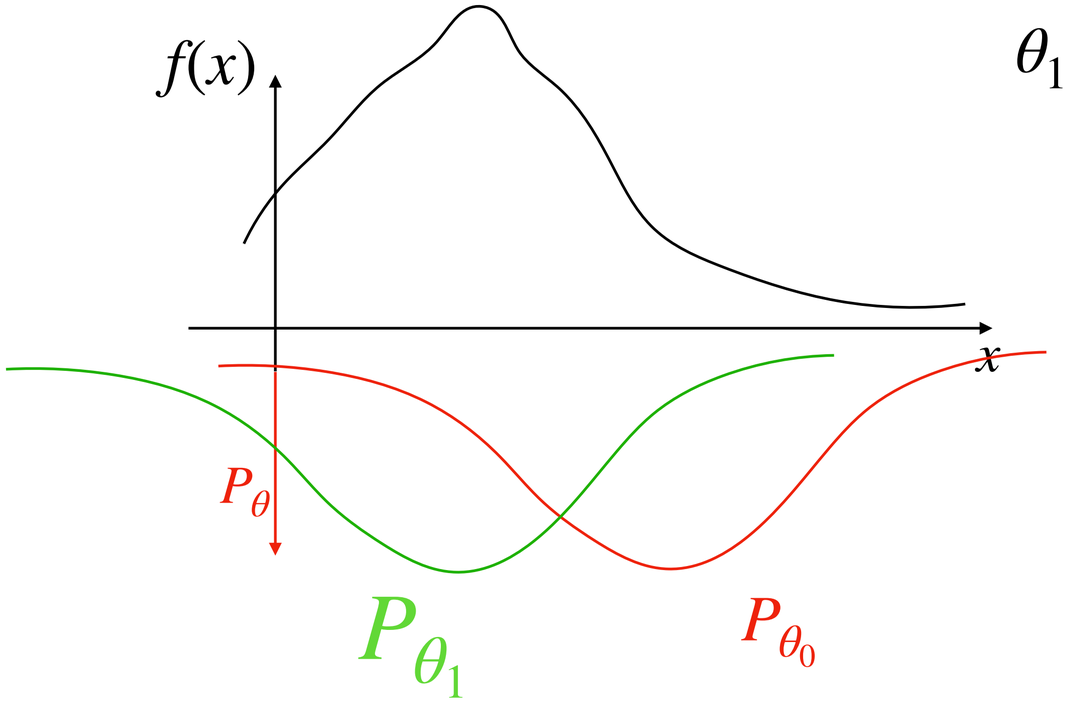


Q: how would this distribution move during the update?

Warm Up:

$$\nabla_\theta J(\theta)\,|_{\theta=\theta_0} = \mathbb{E}_{x\sim P_{\theta_0}} \nabla_\theta \ln P_{\theta_0}(x) \cdot f(x)$$

$$\theta_1 = \theta_0 + \eta \nabla_\theta J(\theta_0)$$



Update distribution (via updating θ) such that P_θ has higher probability mass at regions where $f(x)$ is larger

Q: how would this distribution move during the update?

Outline:



1. Warm up: computing gradient using importance weighting

2. Policy Gradient formulation

Policy Gradient:

Examples of Policy Parameterization (discrete actions)

Recall that we consider parameterized policy $\pi_\theta(\cdot \mid s) \in \Delta(A), \forall s$

1. Softmax Policy for discrete MDPs:

$\theta_{s,a} \in \mathbb{R}, \forall s, a \in S \times A$

Policy Gradient:

Examples of Policy Parameterization (discrete actions)

Recall that we consider parameterized policy $\pi_\theta(\cdot \mid s) \in \Delta(A), \forall s$

1. Softmax Policy for discrete MDPs:

$$\theta_{s,a} \in \mathbb{R}, \forall s, a \in S \times A$$

$$\pi_\theta(a \mid s) = \frac{\exp(\theta_{s,a})}{\sum_{a'} \exp(\theta_{s,a'})}$$

Policy Gradient:

Examples of Policy Parameterization (discrete actions)

Recall that we consider parameterized policy $\pi_\theta(\cdot \mid s) \in \Delta(A), \forall s$

1. Softmax Policy for discrete MDPs:

$$\theta_{s,a} \in \mathbb{R}, \forall s, a \in S \times A$$

$$\pi_\theta(a \mid s) = \frac{\exp(\theta_{s,a})}{\sum_{a'} \exp(\theta_{s,a'})}$$

2. Softmax linear Policy

Policy Gradient:

Examples of Policy Parameterization (discrete actions)

Recall that we consider parameterized policy $\pi_\theta(\cdot \mid s) \in \Delta(A), \forall s$

1. Softmax Policy for discrete MDPs:

$$\theta_{s,a} \in \mathbb{R}, \forall s, a \in S \times A$$

$$\pi_\theta(a \mid s) = \frac{\exp(\theta_{s,a})}{\sum_{a'} \exp(\theta_{s,a'})}$$

2. Softmax linear Policy

Feature vector $\phi(s, a) \in \mathbb{R}^d$, and parameter $\theta \in \mathbb{R}^d$

Policy Gradient:

Examples of Policy Parameterization (discrete actions)

Recall that we consider parameterized policy $\pi_\theta(\cdot \mid s) \in \Delta(A), \forall s$

1. Softmax Policy for discrete MDPs:

$$\theta_{s,a} \in \mathbb{R}, \forall s, a \in S \times A$$

$$\pi_\theta(a \mid s) = \frac{\exp(\theta_{s,a})}{\sum_{a'} \exp(\theta_{s,a'})}$$

2. Softmax linear Policy

Feature vector $\phi(s,a) \in \mathbb{R}^d$, and parameter $\theta \in \mathbb{R}^d$

$$\pi_\theta(a \mid s) = \frac{\exp(\theta^\top \phi(s,a))}{\sum_{a'} \exp(\theta^\top \phi(s,a'))}$$

Policy Gradient:

Examples of Policy Parameterization (discrete actions)

Recall that we consider parameterized policy $\pi_\theta(\cdot \mid s) \in \Delta(A), \forall s$

1. Softmax Policy for discrete MDPs:

$$\theta_{s,a} \in \mathbb{R}, \forall s, a \in S \times A$$

$$\pi_\theta(a \mid s) = \frac{\exp(\theta_{s,a})}{\sum_{a'} \exp(\theta_{s,a'})}$$

2. Softmax linear Policy

Feature vector $\phi(s,a) \in \mathbb{R}^d$, and parameter $\theta \in \mathbb{R}^d$

$$\pi_\theta(a \mid s) = \frac{\exp(\theta^\top \phi(s,a))}{\sum_{a'} \exp(\theta^\top \phi(s,a'))}$$

3. Neural Policy:

Policy Gradient:

Examples of Policy Parameterization (discrete actions)

Recall that we consider parameterized policy $\pi_\theta(\cdot \mid s) \in \Delta(A), \forall s$

1. Softmax Policy for discrete MDPs:

$$\theta_{s,a} \in \mathbb{R}, \forall s, a \in S \times A$$

$$\pi_\theta(a \mid s) = \frac{\exp(\theta_{s,a})}{\sum_{a'} \exp(\theta_{s,a'})}$$

2. Softmax linear Policy

Feature vector $\phi(s,a) \in \mathbb{R}^d$, and parameter $\theta \in \mathbb{R}^d$

$$\pi_\theta(a \mid s) = \frac{\exp(\theta^\top \phi(s,a))}{\sum_{a'} \exp(\theta^\top \phi(s,a'))}$$

3. Neural Policy:

Neural network

$$f_\theta : S \times A \mapsto \mathbb{R}$$

Policy Gradient:

Examples of Policy Parameterization (discrete actions)

Recall that we consider parameterized policy $\pi_\theta(\cdot \mid s) \in \Delta(A), \forall s$

1. Softmax Policy for discrete MDPs:

$$\theta_{s,a} \in \mathbb{R}, \forall s, a \in S \times A$$

$$\pi_\theta(a \mid s) = \frac{\exp(\theta_{s,a})}{\sum_{a'} \exp(\theta_{s,a'})}$$

2. Softmax linear Policy

Feature vector $\phi(s,a) \in \mathbb{R}^d$, and parameter $\theta \in \mathbb{R}^d$

$$\pi_\theta(a \mid s) = \frac{\exp(\theta^\top \phi(s,a))}{\sum_{a'} \exp(\theta^\top \phi(s,a'))}$$

3. Neural Policy:

Neural network $f_\theta : S \times A \mapsto \mathbb{R}$

$$\pi_\theta(a \mid s) = \frac{\exp(f_\theta(s,a))}{\sum_{a'} \exp(f_\theta(s,a'))}$$

Policy Gradient:

Examples of Policy Parameterization (discrete actions)

Recall that we consider parameterized policy $\pi_\theta(\cdot \mid s) \in \Delta(A), \forall s$

1. Softmax Policy for discrete MDPs:

$$\theta_{s,a} \in \mathbb{R}, \forall s, a \in S \times A$$

$$\pi_\theta(a \mid s) = \frac{\exp(\theta_{s,a})}{\sum_{a'} \exp(\theta_{s,a'})}$$

2. Softmax linear Policy

Feature vector $\phi(s,a) \in \mathbb{R}^d$, and parameter $\theta \in \mathbb{R}^d$

$$\pi_\theta(a \mid s) = \frac{\exp(\theta^\top \phi(s,a))}{\sum_{a'} \exp(\theta^\top \phi(s,a'))}$$

3. Neural Policy:

Neural network $f_\theta : S \times A \mapsto \mathbb{R}$

$$\pi_\theta(a \mid s) = \frac{\exp(f_\theta(s,a))}{\sum_{a'} \exp(f_\theta(s,a'))}$$

In high level, think about π_θ as a classifier which has its parameters to be optimized

Derivation of Policy Gradient: REINFORCE

$$\tau = \{s_0, a_0, s_1, a_1, \ldots, s_{H-1}, a_{H-1}\}$$

Derivation of Policy Gradient: REINFORCE

$$\tau = \{s_0, a_0, s_1, a_1, \ldots, s_{H-1}, a_{H-1}\}$$

What's the likelihood of trajectory τ under π_θ?

Derivation of Policy Gradient: REINFORCE

$$\tau = \{s_0, a_0, s_1, a_1, \ldots, s_{H-1}, a_{H-1}\}$$

What's the likelihood of trajectory τ under π_θ?

$$\rho_\theta(\tau) = \mu(s_0)\pi_\theta(a_0 \mid s_0)P(s_1 \mid s_0, a_0)\pi_\theta(a_1 \mid s_1)\ldots$$

Derivation of Policy Gradient: REINFORCE

$$\tau = \{s_0, a_0, s_1, a_1, \ldots, s_{H-1}, a_{H-1}\}$$

What's the likelihood of trajectory τ under π_θ?

$$\rho_\theta(\tau) = \mu(s_0)\pi_\theta(a_0 \mid s_0)P(s_1 \mid s_0, a_0)\pi_\theta(a_1 \mid s_1)\ldots$$

Rewrite the objective using the traj distribution:

$$J(\theta) = \mathbb{E}_{\tau \sim \rho_\theta(\tau)} \underbrace{\left[\sum_{h=0}^{H-1} r(s_h, a_h) \right]}_{R(\tau)}$$

Derivation of Policy Gradient: REINFORCE

$$\tau = \{s_0, a_0, s_1, a_1, \ldots, s_{H-1}, a_{H-1}\}$$

What's the likelihood of trajectory τ under π_θ?

$$\rho_\theta(\tau) = \mu(s_0)\pi_\theta(a_0 \mid s_0)P(s_1 \mid s_0, a_0)\pi_\theta(a_1 \mid s_1)\ldots$$

Rewrite the objective using the traj distribution:

$$J(\theta) = \mathbb{E}_{\tau \sim \rho_\theta(\tau)} \underbrace{\left[\sum_{h=0}^{H-1} r(s_h, a_h) \right]}_{R(\tau)}$$

Let's apply the importance weighting trick!

Derivation of Policy Gradient: REINFORCE

$$J(\theta) = \mathbb{E}_{\tau \sim \rho_\theta(\tau)} \underbrace{\left[\sum_{h=0}^{H-1} r(s_h, a_h) \right]}_{R(\tau)}$$

Let's say we are at θ_0, we want to compute gradient at θ_0, i.e., $\nabla_\theta J(\theta)\,|_{\theta=\theta_0}$

Derivation of Policy Gradient: REINFORCE

$$J(\theta) = \mathbb{E}_{\tau \sim \rho_\theta(\tau)} \underbrace{\left[\sum_{h=0}^{H-1} r(s_h, a_h) \right]}_{R(\tau)}$$

Let's say we are at θ_0, we want to compute gradient at θ_0, i.e., $\nabla_\theta J(\theta)\,|_{\theta=\theta_0}$

$$J(\theta) = \mathbb{E}_{\tau \sim \rho_\theta} R(\tau)$$

Derivation of Policy Gradient: REINFORCE

$$J(\theta) = \mathbb{E}_{\tau \sim \rho_\theta(\tau)} \underbrace{\left[\sum_{h=0}^{H-1} r(s_h, a_h) \right]}_{R(\tau)}$$

Let's say we are at θ_0, we want to compute gradient at θ_0, i.e., $\nabla_\theta J(\theta)\,|_{\theta=\theta_0}$

$$J(\theta) = \mathbb{E}_{\tau \sim \rho_\theta} R(\tau) \;\; = \mathbb{E}_{\tau \sim \rho_{\theta_0}} \left[\frac{\rho_\theta(\tau)}{\rho_{\theta_0}(\tau)} R(\tau) \right]$$

Derivation of Policy Gradient: REINFORCE

$$J(\theta) = \mathbb{E}_{\tau \sim \rho_\theta(\tau)} \underbrace{\left[\sum_{h=0}^{H-1} r(s_h, a_h) \right]}_{R(\tau)}$$

Let's say we are at θ_0, we want to compute gradient at θ_0, i.e., $\nabla_\theta J(\theta)\,|_{\theta=\theta_0}$

$$J(\theta) = \mathbb{E}_{\tau \sim \rho_\theta} R(\tau) \;\; = \mathbb{E}_{\tau \sim \rho_{\theta_0}} \left[\frac{\rho_\theta(\tau)}{\rho_{\theta_0}(\tau)} R(\tau) \right]$$

$$\nabla_\theta J(\theta)\,|_{\theta=\theta_0} = \mathbb{E}_{\tau \sim \rho_{\theta_0}} \left[\frac{\nabla_\theta \rho_\theta(\tau)\,|_{\theta=\theta_0}}{\rho_{\theta_0}(\tau)} R(\tau) \right]$$

Derivation of Policy Gradient: REINFORCE

$$J(\theta) = \mathbb{E}_{\tau \sim \rho_\theta(\tau)} \underbrace{\left[\sum_{h=0}^{H-1} r(s_h, a_h) \right]}_{R(\tau)}$$

Let's say we are at θ_0, we want to compute gradient at θ_0, i.e., $\nabla_\theta J(\theta)\,|_{\theta=\theta_0}$

$$J(\theta) = \mathbb{E}_{\tau \sim \rho_\theta} R(\tau) \;\; = \mathbb{E}_{\tau \sim \rho_{\theta_0}} \left[\frac{\rho_\theta(\tau)}{\rho_{\theta_0}(\tau)} R(\tau) \right]$$

$$\nabla_\theta J(\theta)\,|_{\theta=\theta_0} = \mathbb{E}_{\tau \sim \rho_{\theta_0}} \left[\frac{\nabla_\theta \rho_\theta(\tau)\,|_{\theta=\theta_0}}{\rho_{\theta_0}(\tau)} R(\tau) \right] \;\; = \mathbb{E}_{\tau \sim \rho_{\theta_0}} \left[\nabla_\theta \ln \rho_\theta(\tau)\,|_{\theta=\theta_0} R(\tau) \right]$$

Derivation of Policy Gradient: REINFORCE

$$\nabla_\theta J(\theta)\,|_{\theta=\theta_0} = \mathbb{E}_{\tau\sim\rho_{\theta_0}}\left[\nabla_\theta \ln \rho_\theta(\tau)\,|_{\theta=\theta_0} R(\tau)\right]$$

ρ_θ contains unknown transition, seems that we cannot compute $\nabla \ln \rho_\theta$…

Derivation of Policy Gradient: REINFORCE

$$\nabla_\theta J(\theta)\,|_{\theta=\theta_0} = \mathbb{E}_{\tau\sim\rho_{\theta_0}}\left[\nabla_\theta \ln \rho_\theta(\tau)\,|_{\theta=\theta_0} R(\tau)\right]$$

ρ_θ contains unknown transition, seems that we cannot compute $\nabla \ln \rho_\theta$…

Recall: $\rho_\theta(\tau) = \mu(s_0)\pi_\theta(a_0\,|\,s_0)P(s_1\,|\,s_0, a_0)\pi_\theta(a_1\,|\,s_1)\ldots$

Derivation of Policy Gradient: REINFORCE

$$\nabla_\theta J(\theta)\,|_{\theta=\theta_0} = \mathbb{E}_{\tau\sim\rho_{\theta_0}}\left[\nabla_\theta \ln \rho_\theta(\tau)\,|_{\theta=\theta_0} R(\tau)\right]$$

ρ_θ contains unknown transition, seems that we cannot compute $\nabla \ln \rho_\theta$…

Recall: $\rho_\theta(\tau) = \mu(s_0)\pi_\theta(a_0\,|\,s_0)P(s_1\,|\,s_0, a_0)\pi_\theta(a_1\,|\,s_1)\ldots$

In class excerise: can you plug in the definition of ρ_θ into the above expression, and see if you can simplify the gradient formulation further

Summary so far for Policy Gradients

We derived the most classic PG formulation:

$$\nabla J(\theta) = \mathbb{E}_{\tau \sim \rho_\theta(\tau)} \left[\sum_{h=0}^{H} \nabla_\theta \ln \pi_\theta(a_h \,|\, s_h) \cdot R(\tau) \right]$$

Summary so far for Policy Gradients

We derived the most classic PG formulation:

$$\nabla J(\theta) = \mathbb{E}_{\tau \sim \rho_\theta(\tau)} \left[\sum_{h=0}^{H} \nabla_\theta \ln \pi_\theta(a_h \,|\, s_h) \cdot R(\tau) \right]$$

Increase the likelihood of on trajectories with high total reward

Further simplification on PG

Further simplification on PG

$$\nabla J(\theta) = \mathbb{E}_{\tau \sim \rho_\theta(\tau)} \left[\left(\sum_{h=0}^{H-1} \nabla_\theta \ln \pi_\theta(a_h \mid s_h) \cdot \sum_{\tau=h}^{H-1} r(s_\tau, a_\tau) \right) \right]$$

Further simplification on PG

Further simplification on PG

$$\nabla J(\theta) = \mathbb{E}_{\tau \sim \rho_\theta(\tau)} \left[\left(\sum_{h=0}^{H-1} \nabla_\theta \ln \pi_\theta(a_h \mid s_h) \cdot \sum_{\tau=h}^{H-1} r(s_\tau, a_\tau) \right) \right]$$

(Change action distribution at h only affects rewards later on…)

Further simplification on PG

Further simplification on PG

$$\nabla J(\theta) = \mathbb{E}_{\tau \sim \rho_\theta(\tau)} \left[\left(\sum_{h=0}^{H-1} \nabla_\theta \ln \pi_\theta(a_h \mid s_h) \cdot \sum_{\tau=h}^{H-1} r(s_\tau, a_\tau) \right) \right]$$

(Change action distribution at h only affects rewards later on…)

Exercise:

Show this simplified version is equivalent to REINFORCE

REINFORCE works surprisingly well on generative models

We train LLMs to summarize reddit post

Example Post

SUBREDDIT: r/dogs
TITLE: [HELP] Not sure how to deal with new people/dogs and my big ole pup
POST: I have a three year old Dober/Pit mix named Romulus ("Rome" for short). I live with 3 other dogs: a 10 year old labrador, a 2 year old French Bulldog and a 8 year old maltese mix. The four of them get along just fine, Rome and the Frenchie are best best best best friends. He isn't the best at meeting new people, but not ALWAYS....Then, the crux of the matter: I want to have a 4th of July party. Several people want to bring their dogs. I doubt I can say "no dogs allowed" and I don't want to let everyone else bring their dog and make mine stay at day care all day.
TL;DR:

Summary:

HOW do I introduce new people? HOW do I introduce new dogs? WHAT do I do about 4th of July??

REINFORCE works surprisingly well on generative models

Dataset Composition

- 210K Prompts total
  - 117K Prompts with *Human written summarizations*
  - 93K Prompts with *Human Preference Labels*

REINFORCE works surprisingly well on generative models

Dataset Composition

- 210K Prompts total
  - 117K Prompts with *Human written summarizations*
  - 93K Prompts with *Human Preference Labels*

Used to pre-train a reward model (details will come soon)

Using RL to optimize 7B size model on TL;DR

Model size	Algorithm	Winrate (↑)
6.9B	SFT	45.2 (±2.49)
	DPO	68.4 (±2.01)
	REINFORCE	70.7*
	PPO	77.6‡
	RLOO ($k = 2$)	74.2*
	RLOO ($k = 4$)	<u>77.9*</u>
	REBEL	**78.1** (±1.74)

* directly obtained from Ahmadian et al. (2024)

‡ directly obtained from Huang et al. (2024)

[REBEL: Reinforcement Learning via Regressing Relative Rewards, Neurips 2024]

Using RL to optimize 7B size model on TL;DR

Model size	Algorithm	Winrate (↑)
6.9B	SFT	45.2 (±2.49)
	DPO	68.4 (±2.01)
	REINFORCE	70.7*
	PPO	77.6‡
	RLOO ($k = 2$)	74.2*
	RLOO ($k = 4$)	77.9*
	REBEL	**78.1** (±1.74)

* directly obtained from Ahmadian et al. (2024)

‡ directly obtained from Huang et al. (2024)

Simple REINFORCE already achieves 70% win-rate over humans (evaluated by GPT4)

[REBEL: Reinforcement Learning via Regressing Relative Rewards, Neurips 2024]

Summary for today

1. Importance Weighting Trick

2. Policy Gradient:

REINFORCE (a direct application of our warm up example):

Summary for today

1. Importance Weighting Trick

2. Policy Gradient:

REINFORCE (a direct application of our warm up example):

$$\nabla J(\theta) = \mathbb{E}_{\tau \sim \rho_\theta(\tau)} \left[\left(\sum_{h=0}^{H-1} \nabla_\theta \ln \pi_\theta(a_h \,|\, s_h) \right) R(\tau) \right]$$

Summary for today

1. Importance Weighting Trick

2. Policy Gradient:

REINFORCE (a direct application of our warm up example):

$$\nabla J(\theta) = \mathbb{E}_{\tau \sim \rho_\theta(\tau)} \left[\left(\sum_{h=0}^{H-1} \nabla_\theta \ln \pi_\theta(a_h \,|\, s_h) \right) R(\tau) \right]$$

2. Use unbiased estimate of $\nabla_\theta J(\theta)$, SG ascent converges to local optimal policy

Feedback and Attendance

https://bit.ly/4gYn2nC

